



MAX PLANCK INSTITUTE
FOR DYNAMICS OF COMPLEX
TECHNICAL SYSTEMS
MAGDEBURG



COMPUTATIONAL METHODS IN
SYSTEMS AND CONTROL THEORY

Mixed-precision iterative refinement for low-rank Lyapunov equations

Xiaobo Liu

MPI Magdeburg, Germany

2025 GAMM Annual Meeting,
Poznań University of Technology, Poznań, April 8, 2025

Joint work with Peter Benner



The goal is to evaluate the (continuous-time) **Lyapunov equations**

$$AX + XA^T + W = 0, \quad W = C^T C, \quad A \in \mathbb{R}^{n \times n}, \quad C \in \mathbb{R}^{m \times n}.$$

Numerous applications in **system balancing**, **optimal control**, and **model reduction**, etc., where

- n can be of order $10^4 \sim 10^5$ or beyond, $m \ll n$,
- A is stable ($\operatorname{Re} \lambda_i(A) < 0$ for all i), and
- the solution $X \in \mathbb{R}^{n \times n}$ PSD with **small numerical rank**: for some X_ε with $\operatorname{rank}(X_\varepsilon) \ll n$, $\|X - X_\varepsilon\| \leq u\|X\|$, u is the unit roundoff.

$\Rightarrow$ A full-rank factorization $X = Y^T Y$ exists, the factor Y (often needed in practice) can be well approximated by a **low-rank** matrix (**Komaroff**, '88).



The goal is to evaluate the (continuous-time) **Lyapunov equations**

$$AX + XA^T + W = 0, \quad W = C^T C, \quad A \in \mathbb{R}^{n \times n}, \quad C \in \mathbb{R}^{m \times n}.$$

Numerous applications in **system balancing**, **optimal control**, and **model reduction**, etc., where

- n can be of order $10^4 \sim 10^5$ or beyond, $m \ll n$,
- A is stable ($\operatorname{Re} \lambda_i(A) < 0$ for all i), and
- the solution $X \in \mathbb{R}^{n \times n}$ PSD with **small numerical rank**: for some X_ε with $\operatorname{rank}(X_\varepsilon) \ll n$, $\|X - X_\varepsilon\| \leq u\|X\|$, u is the unit roundoff.

$\Rightarrow$ A full-rank factorization $X = Y^T Y$ exists, the factor Y (often needed in practice) can be well approximated by a **low-rank** matrix (**Komaroff, '88**).



Newton iteration for the factored solution Y : (Benner & Quintana-Ortí, '99)

$$A_{k+1} = \frac{1}{2}(\mu_k A_k + \mu_k^{-1} A_k^{-1}), \quad A_0 = A \in \mathbb{R}^{n \times n},$$

$$C_{k+1} = \sqrt{\frac{\mu_k}{2}} \begin{bmatrix} C_k \\ \mu_k^{-1} C_k A_k^{-T} \end{bmatrix}, \quad C_0 = C \in \mathbb{R}^{r_0 \times n} \text{ (short \& fat),}$$

with $A_k \rightarrow -I_n$ and $C_k \rightarrow \sqrt{2}Y$ quadratically as $k \rightarrow \infty$.

- In each iter., **no. of rows of C_k doubled**: $r_k = 2^k r_0$.
- **Resolution**: Compute RRQR: (Benner & Quintana-Ortí, '99)

$$\begin{aligned} C_{k+1} \Pi_{k+1} &= Q_{k+1} R_{k+1} =: [U_{k+1} \quad V_{k+1}] \begin{bmatrix} T_{k+1} & F_{k+1} \\ 0 & S_{k+1} \end{bmatrix} \\ &= U_{k+1} [T_{k+1} \quad F_{k+1}] + [0 \quad V_{k+1} S_{k+1}], \end{aligned}$$

and uses a truncated (permuted) R-factor in the next iter.



Newton iteration for the factored solution Y : (Benner & Quintana-Ortí, '99)

$$A_{k+1} = \frac{1}{2}(\mu_k A_k + \mu_k^{-1} A_k^{-1}), \quad A_0 = A \in \mathbb{R}^{n \times n},$$

$$C_{k+1} = \sqrt{\frac{\mu_k}{2}} \begin{bmatrix} C_k \\ \mu_k^{-1} C_k A_k^{-T} \end{bmatrix}, \quad C_0 = C \in \mathbb{R}^{r_0 \times n} \text{ (short \& fat),}$$

with $A_k \rightarrow -I_n$ and $C_k \rightarrow \sqrt{2}Y$ quadratically as $k \rightarrow \infty$.

- In each iter., **no. of rows of C_k doubled**: $r_k = 2^k r_0$.
- **Resolution**: Compute RRQR: (Benner & Quintana-Ortí, '99)

$$\begin{aligned} C_{k+1} \Pi_{k+1} &= Q_{k+1} R_{k+1} =: [U_{k+1} \quad V_{k+1}] \begin{bmatrix} T_{k+1} & F_{k+1} \\ 0 & S_{k+1} \end{bmatrix} \\ &= U_{k+1} [T_{k+1} \quad F_{k+1}] + [0 \quad V_{k+1} S_{k+1}], \end{aligned}$$

and uses a truncated (permuted) R-factor in the next iter.



The **tailing block** S_{k+1} in the R-factor can be safely dropped if

$$\|S_{k+1}\|_2 \leq \sqrt{u} \|R_{k+1}\|_2.$$

For $\tilde{C}_{k+1} := [T_{k+1} \ F_{k+1}] \Pi_{k+1}^T$ and $\widetilde{W}_{k+1} := \tilde{C}_{k+1}^T \tilde{C}_{k+1}$,

$$\begin{aligned} \|W_{k+1} - \widetilde{W}_{k+1}\|_2 &= \left\| \Pi_{k+1} \left(R_{k+1}^T R_{k+1} - \begin{bmatrix} T_{k+1}^T \\ F_{k+1}^T \end{bmatrix} [T_{k+1} \ F_{k+1}] \right) \Pi_{k+1}^T \right\|_2 \\ &= \left\| \begin{bmatrix} T_{k+1}^T & 0 \\ F_{k+1}^T & S_{k+1}^T \end{bmatrix} \begin{bmatrix} T_{k+1} & F_{k+1} \\ 0 & S_{k+1} \end{bmatrix} - \begin{bmatrix} T_{k+1}^T \\ F_{k+1}^T \end{bmatrix} [T_{k+1} \ F_{k+1}] \right\|_2 \\ &= \|S_{k+1}^T S_{k+1}\|_2 = \|S_{k+1}\|_2^2 \\ &\leq u \|R_{k+1}\|_2^2 = u \|C_{k+1}\|_2^2 = u \|W_{k+1}\|_2. \end{aligned}$$

- Using $\tilde{C}_{k+1}$ in the iter. incurs $O(u)$ **relative perturbation** in the 2-norm when recovering the full solution W_{k+1} .



The **tailing block** S_{k+1} in the R-factor can be safely dropped if

$$\|S_{k+1}\|_2 \leq \sqrt{u} \|R_{k+1}\|_2.$$

For $\tilde{C}_{k+1} := [T_{k+1} \ F_{k+1}] \Pi_{k+1}^T$ and $\tilde{W}_{k+1} := \tilde{C}_{k+1}^T \tilde{C}_{k+1}$,

$$\begin{aligned} \|W_{k+1} - \tilde{W}_{k+1}\|_2 &= \left\| \Pi_{k+1} \left(R_{k+1}^T R_{k+1} - \begin{bmatrix} T_{k+1}^T \\ F_{k+1}^T \end{bmatrix} [T_{k+1} \ F_{k+1}] \right) \Pi_{k+1}^T \right\|_2 \\ &= \left\| \begin{bmatrix} T_{k+1}^T & 0 \\ F_{k+1}^T & S_{k+1}^T \end{bmatrix} \begin{bmatrix} T_{k+1} & F_{k+1} \\ 0 & S_{k+1} \end{bmatrix} - \begin{bmatrix} T_{k+1}^T \\ F_{k+1}^T \end{bmatrix} [T_{k+1} \ F_{k+1}] \right\|_2 \\ &= \|S_{k+1}^T S_{k+1}\|_2 = \|S_{k+1}\|_2^2 \\ &\leq u \|R_{k+1}\|_2^2 = u \|C_{k+1}\|_2^2 = u \|W_{k+1}\|_2. \end{aligned}$$

- Using $\tilde{C}_{k+1}$ in the iter. incurs $O(u)$ **relative perturbation** in the 2-norm when recovering the full solution W_{k+1} .



Newton iteration for factored solution

The full algorithm

Solve $AX + XA^T + C^T C = 0$ for $X = Y^T Y$: (Benner & Quintana-Ortí, '05)

```
1 while  $\|A + I_n\|_1 > \tau$  do
2    $P = A$ 
3    $A = \frac{1}{2}(\mu P + \mu^{-1} P^{-1})$  [*] with  $\mu \leftarrow \sqrt{\|P^{-1}\|_F / \|P\|_F}$ 
4    $C \leftarrow \sqrt{\frac{\mu}{2}} \begin{bmatrix} C \\ \mu^{-1} C P^{-T} \end{bmatrix}$ 
5   if  $\text{size}(C, 1) > \rho n$  then
6     Compute QRCP  $C = Q \begin{bmatrix} T & F \\ 0 & S \end{bmatrix} \Pi^T$  with  $\|S\|_2 \leq \varepsilon \|\tilde{C}\|_2$ .
7      $C \leftarrow \begin{bmatrix} T & F \end{bmatrix} \Pi^T$ 
8  $Y = C / \sqrt{2}$ 
9 return  $Y$ 
```

- Parameters: cge. tol: $\tau = 10\sqrt{nu}$, rank trunc. tol: $\varepsilon = \sqrt{u}$, size tol $\rho \ll 1$
- [*] Dominant cost in each iter.



Mixed-precision truncation scheme

Within the Newton iteration

Core idea: store block rows of truncated R-factor in different precisions,

$$C_k = [T_k \ F_k] \Pi_k^T = \begin{bmatrix} R_1^{(k)} \\ R_2^{(k)} \\ \vdots \\ R_p^{(k)} \end{bmatrix} \Pi_k^T =: \begin{bmatrix} \tilde{R}_1^{(k)} \\ \tilde{R}_2^{(k)} \\ \vdots \\ \tilde{R}_p^{(k)} \end{bmatrix},$$

$\tilde{R}_i^{(k)}$ stored in precision u_i . In next iter.,

$$C_{k+1} = \sqrt{\frac{\mu_k}{2}} \begin{bmatrix} C_k \\ \mu_k^{-1} C_k A_k^{-T} \end{bmatrix}, \quad \text{where } C_k A_k^{-T} = \begin{bmatrix} \tilde{R}_1^{(k)} A_k^{-T} \\ \tilde{R}_2^{(k)} A_k^{-T} \\ \vdots \\ \tilde{R}_p^{(k)} A_k^{-T} \end{bmatrix}.$$

- Each of the p mat mul. done in precision u_i , $i = 1 : p$.
- The mp. block row format can be maintained in *all* following iterations.



Let C' represent the C with mixed-precision block rows:

$$C' = \begin{bmatrix} \tilde{R}_1 + \Delta\tilde{R}_1 \\ \tilde{R}_2 + \Delta\tilde{R}_2 \\ \vdots \\ \tilde{R}_p + \Delta\tilde{R}_p \end{bmatrix}, \quad \|\Delta\tilde{R}_i\| \leq u_i \|\tilde{R}_i\|,$$

Then $\Delta X := C'^T C' - C^T C = \sum_{i=1}^p (\tilde{R}_i^T \Delta\tilde{R}_i + \tilde{R}_i \Delta\tilde{R}_i^T + \Delta\tilde{R}_i^T \Delta\tilde{R}_i)$.
Choose precision u_i by

$$u_i \leq \frac{\|C^T C\|}{\|\tilde{R}_i\|^2} u \approx \frac{\|X\|}{\|\tilde{R}_i\|^2} u, \quad \frac{\|X\|}{\|\tilde{R}_i\|^2} \geq 1,$$

then

$$\frac{\|\Delta X\|}{\|X\|} \leq \frac{2 \sum_{i=1}^p u_i \|\tilde{R}_i\|^2}{\|X\|} + O(u_p^2) \leq 2pu + O(u_p^2).$$

- **Implication:** Lower precision u_i can be used for the block row R_i of smaller norm.



For systems of linear equations $Ax = b$, using potentially three precisions $u_r \leq u \leq u_s$: (**Carson & Higham, '18**)

- u , the working precision
- u_s , the precision for solving the correction equation
- u_r , the precision for residual computation

-
- 1 Solve $Ax = b$ in precision u_s and store x at precision u
 - 2 **repeat**
 - 3 Compute $r = b - Ax$ at precision u_r and round r to precision u_s
 - 4 Solve $Ad = r$ at precision u_s and store d at precision u
 - 5 $x \leftarrow x + d$ at precision u .
 - 6 **until** convergence
-



Solve $AX + XA^T + C^T C = 0$ for $X = Y^T Y$ using the mixed-precision iterative refinement:

-
- 1 Solve $AX + XA^T + C^T C = 0$ in precision u_s and store the solution factor Y at precision u
 - 2 **repeat**
 - 3 Compute $\mathcal{R}(Y) := AY^T Y + Y^T Y A^T + C^T C \approx C_+^T C_+ - C_-^T C_-$ at precision u_r and round $\mathcal{R}(Y)$ to precision u_s ► *Factorization of the symmetric indefinite residual $\mathcal{R}(Y)$ into factored PSD + NSD parts*
 - 4 Solve $AX_+ + X_+ A^T + C_+^T C_+ = 0$ and $AX_- + X_- A^T + C_-^T C_- = 0$ at precision u_s and store solution factors Y_+ and Y_- at precision u
 - 5 Find factor Y of the nearest PSD matrix to $X \leftarrow X + X_+ - X_-$ at precision u . ► *RHS not necessarily PSD, Y obtained without forming X*
 - 6 **until convergence**
-



Solve $AX + XA^T + C^T C = 0$ for $X = Y^T Y$ using the mixed-precision iterative refinement:

-
- 1 Solve $AX + XA^T + C^T C = 0$ in precision u_s and store the solution factor Y at precision u
 - 2 **repeat**
 - 3 Compute $\mathcal{R}(Y) := AY^T Y + Y^T Y A^T + C^T C \approx C_+^T C_+ - C_-^T C_-$ at precision u_r and round $\mathcal{R}(Y)$ to precision u_s ► *Factorization of the symmetric indefinite residual $\mathcal{R}(Y)$ into factored PSD + NSD parts*
 - 4 Solve $AX_+ + X_+ A^T + C_+^T C_+ = 0$ and $AX_- + X_- A^T + C_-^T C_- = 0$ at precision u_s and store solution factors Y_+ and Y_- at precision u
 - 5 Find factor Y of the nearest PSD matrix to $X \leftarrow X + X_+ - X_-$ at precision u . ► *RHS not necessarily PSD, Y obtained without forming X*
 - 6 **until convergence**
-



Solve $AX + XA^T + C^T C = 0$ for $X = Y^T Y$ using the mixed-precision iterative refinement:

-
- 1 Solve $AX + XA^T + C^T C = 0$ in precision u_s and store the solution factor Y at precision u
 - 2 **repeat**
 - 3 Compute $\mathcal{R}(Y) := AY^T Y + Y^T Y A^T + C^T C \approx C_+^T C_+ - C_-^T C_-$ at precision u_r and round $\mathcal{R}(Y)$ to precision u_s $\blacktriangleright$ *Factorization of the symmetric indefinite residual $\mathcal{R}(Y)$ into factored PSD + NSD parts*
 - 4 Solve $AX_+ + X_+ A^T + C_+^T C_+ = 0$ and $AX_- + X_- A^T + C_-^T C_- = 0$ at precision u_s and store solution factors Y_+ and Y_- at precision u
 - 5 Find factor Y of the *nearest* PSD matrix to $X \leftarrow X + X_+ - X_-$ at precision u . $\blacktriangleright$ *RHS not necessarily PSD, Y obtained without forming X*
 - 6 **until** *convergence*
-



$$\begin{aligned}\mathcal{R}(Y) &:= AY^TY + Y^TYA^T + C^TC = \begin{bmatrix} AY^T & Y^T & C^T \end{bmatrix} \begin{bmatrix} Y \\ YA^T \\ C \end{bmatrix} \\ &= \begin{bmatrix} Y^T & (YA^T)^T & C^T \end{bmatrix} \begin{bmatrix} 0 & I & 0 \\ I & 0 & 0 \\ 0 & 0 & I \end{bmatrix} \begin{bmatrix} Y \\ YA^T \\ C \end{bmatrix} =: FPF^T, \text{ } F \text{ tall and skinny.}\end{aligned}$$

Compute *thin* QR: $UT = F$ and spectral decomposition: $TPT^T = Q\Lambda Q^T$.
With *truncation*, (Benner et al., '11)

$$\mathcal{R}(Y) \approx U(Q_+\Lambda_+Q_+^T + Q_-\Lambda_-Q_-^T)U^T = C_+^TC_+ - C_-^TC_-,$$

where

$$C_+ = \sqrt{\Lambda_+}Q_+^TU^T, \quad C_- = \sqrt{-\Lambda_-}Q_-^TU^T.$$



$$\begin{aligned} X_{\text{new}} &:= X_{\text{old}} + X_+ - X_- = \begin{bmatrix} Y^T & Y_+^T & Y_-^T \end{bmatrix} \begin{bmatrix} Y \\ Y_+ \\ -Y_- \end{bmatrix} \\ &= \begin{bmatrix} Y^T & Y_+^T & Y_-^T \end{bmatrix} \begin{bmatrix} I & 0 & 0 \\ 0 & I & 0 \\ 0 & 0 & -I \end{bmatrix} \begin{bmatrix} Y \\ Y_+ \\ Y_- \end{bmatrix} =: FPF^T, \quad F \text{ tall and skinny.} \end{aligned}$$

Compute *thin* QR: $UT = F$ and spectral decomposition: $TPT^T = Q\Lambda Q^T$.
With *truncation*, (Benner et al., '11)

$$X_{\text{new}} \approx U(Q_+\Lambda_+Q_+^T)U^T = Y_{\text{new}}^T Y_{\text{new}},$$

where

$$Y_{\text{new}} = \sqrt{\Lambda_+} Q_+^T U^T.$$



Dominant cost in each refinement step:

- Solve $AX_+ + X_+A^T + C_+^TC_+ = 0$ and $AX_- + X_-A^T + C_-^TC_- = 0$ at precision u_s .
- A unified Newton iteration for two correction equations *simultaneously*:

⋮

$$1 \quad A \leftarrow \frac{1}{2}(\mu P + \mu^{-1}P^{-1}) \text{ with } \mu \leftarrow \sqrt{\|P^{-1}\|_F / \|P\|_F}$$

$$2 \quad C_+ \leftarrow \sqrt{\frac{\mu}{2}} [C_+^T \quad \mu^{-1}P^{-1}C_+^T]^T$$

$$3 \quad C_- \leftarrow \sqrt{\frac{\mu}{2}} [C_-^T \quad \mu^{-1}P^{-1}C_-^T]^T$$

$$4 \quad \text{if } \text{size}(C_+, 1) > \rho n \text{ then } C_+ = \text{RANKTRUNC}(C_+)$$

$$5 \quad \text{if } \text{size}(C_-, 1) > \rho n \text{ then } C_- = \text{RANKTRUNC}(C_-)$$

⋮

- Saves a full $n \times n$ matrix inversion in each Newton iteration.
- Total asymptotic cost: $2(\ell_0 + \ell_1 + \dots + \ell_k)n^3$ flops in precision u_s . ℓ_i the number of simultaneous Newton iterations at i th refinement step.



Table: Parameters for bf16, fp16, fp32, and fp64 arithmetic.

	t	e	u	$x_{\min}$	$x_{\max}$
bf16	8	8	3.91×10^{-3}	1.18×10^{-38}	3.39×10^{38}
fp16	11	5	4.88×10^{-4}	6.10×10^{-5}	6.55×10^4
fp32	24	8	5.96×10^{-8}	1.18×10^{-38}	3.40×10^{38}
fp64	53	11	1.11×10^{-16}	2.22×10^{-308}	1.80×10^{308}

Table: Approximate condition number bounds and limiting residuals, provided that the Newton iteration solver is *numerically stable*. (Carson & Higham, '18)

u_s	u	u_r	Condition bound	Limiting residual
fp16/bf16	fp32	fp32	$10^4/10^3$	fp32
fp16/bf16		fp64	$10^4/10^3$	
fp32		fp64	10^8	
fp16/bf16	fp64	fp64	$10^4/10^3$	fp64
fp32		fp64	10^8	

- A cost balance: lower precision $u_s \implies$ slower convergence of Newton iteration, requiring more iterations ℓ_i (at i th refinement step).



Table: Results of the iterative refinement scheme with $u_r = u$. Results with $u_r < u$ are similar. $m = 3$.

		$u = \text{fp32}$						$u = \text{fp64}$								
n	cond	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
		Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
100	1.3e0	1.7e-6	4	6	6.4e-7	2	6	5.2e-16	16	15	3.9e-15	6	15	1.1e-16	4	15
	3.2e0	5.8e-7	6	9	3.7e-7	3	11	7.2e-15	16	24	3.6e-15	9	24	5.8e-17	5	24
	1.0e1	8.0e-7	8	11	3.1e-8	4	16	6.4e-16	22	33	9.3e-17	12	33	1.8e-15	5	33
	3.2e1	5.4e-6	10	13	3.0e-7	4	19	2.8e-15	38	41	3.7e-15	12	44	7.2e-17	6	41
	1.0e2	4.4e-6	18	14	5.9e-8	5	22	6.6e-15	90	49	9.2e-16	15	49	6.0e-17	6	49
	3.2e2	2.8e-6	30	17	4.1e-8	5	25	7.7e-15	72	59	5.9e-16	15	56	6.8e-17	7	56
	1.0e3	2.5e-4	-	-	6.4e-8	5	27	1.9e-4	-	-	8.7e-16	15	65	6.7e-17	7	63
	3.2e3	8.4e-4	-	-	8.7e-8	5	28	5.4e-4	-	-	7.8e-16	15	70	9.0e-17	7	69
1000	1.3e0	3.0e-7	4	6	2.4e-7	2	6	8.5e-14	16	45	1.3e-15	6	12	1.5e-17	4	12
	3.2e0	1.1e-6	4	6	1.3e-7	3	6	4.9e-14	20	59	1.8e-15	9	22	1.4e-17	5	22
	1.0e1	1.2e-5	4	9	7.3e-6	3	9	1.7e-14	26	31	9.6e-15	12	31	8.6e-16	5	31
	3.2e1	2.7e-5	6	10	1.2e-7	4	10	2.7e-5	-	-	2.1e-15	12	39	2.9e-16	6	39
	1.0e2	7.6e-5	-	-	5.7e-7	4	12	7.7e-5	-	-	1.6e-16	16	48	2.1e-16	6	48
	3.2e2	2.0e-4	-	-	6.0e-9	5	12	2.0e-4	-	-	1.6e-16	15	55	2.8e-16	7	55
	1.0e3	1.1e-4	-	-	6.9e-9	5	12	1.1e-4	-	-	1.3e-15	15	71	2.1e-16	7	61
	3.2e3	5.0e-5	18	13	9.8e-9	5	11	3.4e-6	-	-	2.8e-14	15	99	3.1e-16	7	69



Table: Results on problems of larger condition numbers.

n	cond	$u = \text{fp64}, u_s = \text{fp32}$			$u = \text{fp64}, u_s = \text{fp64}$		
		Res	iter	rank	Res	iter	rank
100	1.0e4	4.9e-15	18	75	1.5e-16	7	74
	1.0e5	2.7e-15	24	83	1.4e-15	8	82
	1.0e6	1.7e-15	36	88	4.5e-15	8	87
	1.0e7	3.1e-15	72	90	6.7e-15	8	90
	1.0e8	4.8e-7	-	-	9.5e-17	16	88
1000	1.0e4	3.5e-16	20	79	4.2e-16	7	75
	1.0e5	1.2e-14	30	153	5.3e-16	7	85
	1.0e6	3.0e-14	42	277	6.3e-16	8	94
	1.0e7	4.2e-14	108	325	4.2e-15	8	101
	1.0e8	7.2e-5	-	-	5.6e-16	16	114

- **Convergence** for $\text{cond} \lesssim 10^2$ with $u_s = \text{bf16}$, for $\text{cond} \lesssim 10^7$ with $u_s = \text{fp32}$.
- The **iter**, incl. ℓ_h , ℓ_s , and ℓ_d for $u_s = \text{bf16}$, fp32 , and fp64 , increases as **cond** grows. The ratios ℓ_h/ℓ_s and ℓ_s/ℓ_d are **approx. 2** for **well conditioned** problems. (Up to $10^{1.5}$ for ℓ_h/ℓ_s and up to 10^4 for ℓ_s/ℓ_d .)
- Potential speedup as fp16 (bf16) can be theoretically **8×** or **16×** faster than fp32 or fp64 through NVIDIA's Tensor Cores (**Haidar, Tomov, & Dongarra, '19**).



Table: Summary of test problems from the SLICOT library (Chahlaoui & Dooren, '02).

Dataset	n	m	Nonzeros	cond_F	$\text{cond}_{\text{sign}}$
beam	348	1	60,726	1.2e7	1.4e9
build	48	1	1,176	7.5e4	2.0e6
CDplayer	120	2	240	1.4e5	4.1e10
eady	598	1	357,406	3.6e3	3.8e6
fom	1,006	1	1,012	2.3e4	5.2e5
heat-cont	200	1	598	1.5e5	1.0e4
iss	270	3	405	2.5e5	1.4e11
pde	84	1	382	1.2e2	1.6e2
random	200	1	2132	3.2e3	2.3e10

- The $\text{cond}_{\text{sign}}$ can probably give information to *stability* of Newton iteration in floating-point arithmetic (Higham, '08).



Table: Results on test problems from the SLICOT library.

	$u = \text{fp32}$						$u = \text{fp64}$								
Dataset	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
beam	1.8e-3	-	-	3.2e-7	14	73	8.7e-4	-	-	3.2e-7	-	-	7.1e-16	16	134
build	2.0e-4	-	-	7.8e-8	14	39	2.1e-4	-	-	5.5e-17	70	48	2.4e-17	15	48
CDplayer	4.3e-6	8	2	1.4e-9	16	23	5.9e-8	-	-	3.6e-16	48	116	1.9e-16	18	116
eady	2.1e-3	-	-	1.3e-7	15	12	2.3e-3	-	-	2.7e-14	45	106	2.3e-16	16	88
fom	2.4e-3	-	-	6.5e-9	13	16	2.4e-3	-	-	7.2e-16	39	30	7.8e-17	15	27
heat-cont	4.2e-4	-	-	2.7e-8	7	12	4.1e-4	-	-	2.1e-15	28	28	4.5e-17	9	26
iss	6.5e-6	30	12	2.0e-8	21	119	6.5e-6	-	-	2.0e-8	-	-	4.9e-17	23	223
pde [*]	7.4e-7	8	5	2.5e-8	4	6	1.0e-15	22	11	8.0e-17	12	11	9.7e-17	6	11
random	7.2e-4	-	-	6.9e-8	15	7	7.6e-4	-	-	2.7e-9	-	-	1.3e-16	16	24

[*] Problem of $\text{cond} \leq 10^3$.

- Convergence only on *well conditioned* problems with $u_s = \text{bf16}$.
- Divergence with $u_s = \text{fp32}$ on three problems with $\text{cond}_{\text{sign}} \sim 10^{10}$.



- A formalized mixed-precision iterative refinement framework for low-rank Lyapunov equations $AX + XA^T + C^T C = 0$.
- Improved solver by combining the two Newton iterations for solving the **correction equations** into one.
- A new mixed-precision rank-truncation scheme, potentially useful when A is “easy” to invert.
- Reduced precisions, such as the **half precision**, can be used to accelerate the solution of *well conditioned* Lyapunov equations, say, those of $\text{cond} \leq 1/\sqrt{u_s}$.

► P. Benner and X. Liu. Mixed-precision iterative refinement for low-rank Lyapunov equations. preprint, [https:// ongoing work, coming soon](https://ongoingwork.comingsoon).

Thank you for your attention!



Peter Benner, Pablo Ezzatti, Daniel Kressner, Enrique S. Quintana-Ortí, and Alfredo Remón. (Benner et al.)

A mixed-precision algorithm for the solution of Lyapunov equations on hybrid CPU–GPU platforms.

Parallel Comput., 37:439–450, 2011.



Peter Benner and Enrique S. Quintana-Ortí.

Solving stable generalized Lyapunov equations with the matrix sign function.

Numer. Algorithms, 20:75–100, 1999.



Peter Benner and Enrique S. Quintana-Ortí.

Model reduction based on spectral projection methods.

In *Dimension Reduction of Large-Scale Systems*, Lecture Notes in Computational Science and Engineering, vol 45, Springer, Berlin, Heidelberg, 2005.



Erin Carson and Nicholas J. Higham.

Accelerating the solution of linear systems by iterative refinement in three precisions.

SIAM J. Sci. Comput., 40(2):A817–A847, 2018.



Younès Chahlaoui and Paul Van Dooren.

A collection of benchmark examples for model reduction of linear time invariant dynamical systems.

SLICOT Working Note 2002-2: February 2002.



Azzam Haidar, Stanimire Tomov, and Jack Dongarra.

Towards half-precision computation for complex matrices: A case study for mixed precision solvers on GPUs.

In *2019 IEEE/ACM 10th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA)*, Denver, CO, USA, 2019, pages 17–24. IEEE.



Nicholas J. Higham.

Functions of Matrices: Theory and Computation.

Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2008. xx+425 pp. ISBN 978-0-898716-46-7.



N. Komaroff.

Simultaneous eigenvalue lower bounds for the Lyapunov matrix equation.

IEEE Trans. Automat. Control, 33(1):126–128, 1988.